

Master Business Rules and Audit Specification

How money, stock and records work in NairaPlate

Version 1.2, 3 October 2026

Describes The live system after the ledger programme (steps 1 to 5 and 4b) and the security sweep findings

Readers Parts 1 to 11: owners, accountants, managers. Appendices A to H: engineers and auditors

Database 45 tables, 78 access rules, 79 functions, 36 triggers, read from the live database on 2 October 2026

Status Reference manual. Sign-off is not yet possible: see Part 11

Contents

1. NairaPlate: Master Business Rules and Audit Specification
2. Part 1. The principles behind every money rule
3. Part 2. Roles: who can do what
4. Part 3. Area by area: the rules
5. Part 4. How the key numbers are worked out
6. Part 5. Audit: what is recorded
7. Part 6. Every kind of money event, accounted for
8. Part 7. The security model in plain words
9. Part 8. Evidence: what has actually been proved
10. Part 9. Known gaps and open items
11. Part 10. How the ledger programme got here
12. Part 11. Sign-off checklist for the security and safety sweep
13. Technical appendices
14. Appendix A. Tables and who can write to them
15. Appendix B. Functions callable from the app
16. Appendix C. Triggers
17. Appendix D. Ledger rules at a glance
18. Appendix E. Server routes
19. Appendix F. Audit events
20. Appendix G. SQL files for the programme
21. Appendix H. Glossary

NairaPlate: Master Business Rules and Audit Specification

Version 1.2, 3 October 2026

Version 1.2 corrects Part 4.5 (the sale-time cost snapshot already existed and was described wrongly in 1.1) and adds the owner corrections for channel payouts, price decisions and batches (Step 7): the database functions are live and rehearsed; the three screens are released but **have not yet been used by a person** (Part 8.3). Version 1.1 recorded the sweep results: the batch fault (F0), the table locks A1 to A4 and the permission clean-up (B1 to B3) are now applied on the live database. Version 1.0 (2 October 2026) described them as open.

This is the reference for how money, stock and records work in NairaPlate. It is written in two layers.

- **Parts 1 to 9** are for owners, accountants and managers. They are in plain language: what the rules are, who can do what, how a mistake is corrected, how the numbers are worked out, and what the system does and does not record.
- **Appendices A to I** are for engineers and auditors. They list the tables, access rules, functions, triggers, constraints, server routes and audit events, as read from the live database on 2 and 3 October 2026.

How this document was produced, and how far to trust it

- The facts about the database (tables, access rules, functions, triggers) were read from the live database on 2 and 3 October 2026, not written from memory.
- The formulas were read from the code (`src/lib`) and from the database functions.
- Every rule is marked with how it was proved, using these codes:
- **[C]** read from the live database or code.
- **[L]** tested on a local copy of the database during the build.
- **[R]** tested on the live database in a rehearsal (2 and 3 October 2026) that always undoes itself (Part 8).
- **[S]** the screen has not yet been used by a person in the Demo Kitchen test. A rule can be proved at the database level and still be unproven on the screen.
- Where something is not proved, or does not work, this document says so. Part 9 lists every known gap in one place.
- Money is stored in **kobo** (whole numbers). 100 kobo is ₦1. Times are Nigeria time (UTC+1).

Part 1. The principles behind every money rule

1. **Entries are added, never changed.** A payment, a write-off, a purchase, a refund or a count is a line in a record. Nobody edits or deletes that line afterwards. This applies to catering payments, customer credit, supplier entries, purchases, closed cash drawers and audit entries.
2. **A mistake is corrected by a new line beside the old one.** The correction is called a **reversal** (or an **adjustment** for a drawer count). The original stays on screen, struck through, with the reason for the correction next to it.
3. **Only owners reverse.** Reversals, write-offs, adjustments and closing someone else's shift need an owner (or a Supa Admin), and every one needs a **reason of at least 5 characters**.
4. **One home for every number.** Each figure that matters (expected cash, what a customer owes, what you owe a supplier, profit) is worked out in exactly one place. Every screen and report calls that place, so two screens cannot disagree.
5. **The database enforces the rules, not only the screens.** A screen hides a button. The database refuses the action. If the two ever disagree, the database wins.
6. **Money records are written only by approved functions.** The browser cannot insert or edit a sale, a payment, a purchase, a stock figure or a closed shift directly. It has to ask a database function that checks who is asking, which business they belong to, and whether the plan is active.
7. **Each business sees only its own records.** This is checked on every table and in every function, using the business in the signed-in person's login token.
8. **Nothing is guaranteed to be perfect.** These controls make fraud and mistakes hard and visible. They do not make them impossible. In particular, they cannot know whether a customer really paid, or whether cash was really counted. They can only make sure that what was recorded cannot be quietly changed.

Part 2. Roles: who can do what

NairaPlate has six roles. A person's role is set by the server when they sign in and is carried in their login token. It cannot be changed from the browser.

Role	In plain words
Owner	Runs the business. Can do everything for their own business, including corrections.
Supa Admin	A second manager. Treated the same as an owner for every action in this document. Only a Supa Admin can give or remove the Supa Admin role.
Cashier	Takes sales and payments at the till and runs the cash drawer. Cannot correct records.
Purchaser	Buys ingredients, records supplier payments, counts stock. Cannot correct records.
Cook	Logs batches and wastage, counts stock.
Platform Admin	NairaPlate staff. Approves businesses, manages plans and settings. Not part of any business.

2.1 Action matrix

"Yes" means the database allows it. A dash means it refuses. [C][R]

Action	Owner / Supa Admin	Cashier	Purchaser	Cook
Take a cash, split, transfer or credit sale	Yes	Yes	No	No
Void or refund a sale	Yes (any sale)	Yes, own sales only	No	No
Record a payment on a customer debt	Yes	Yes	No	No
Write off a debt, reverse a debt entry, add a debt by hand	Yes	No	No	No
Create a catering order, record a catering payment, change catering status	Yes	Yes	No	No
Reverse a catering payment	Yes	No	No	No
Log a purchase	Yes	No	Yes	No
Reverse a purchase	Yes	No	No	No
Change an ingredient's price	Yes	No	Yes	No
Record a supplier payment	Yes	No	Yes	No
Reverse a supplier payment	Yes	No	No	No
Log wastage	Yes	No	Yes	Yes
Log a batch	Yes	No	No	Yes

Action	Owner / Supa Admin	Cashier	Purchaser	Cook
Submit a stock count	Yes (applies at once)	No	Yes (waits for an owner)	Yes (waits for an owner)
Approve or reject a stock count	Yes	No	No	No
Record the opening stock count	Yes	No	No	No
Open a cash drawer shift	Yes	Yes	No	No
Close their own shift	Yes	Yes	No	No
Close a shift someone else left open	Yes	No	No	No
Correct the count on a closed shift	Yes	No	No	No
Record a channel payout, decide a price	Yes	No	No	No
Edit recipes	Yes	No	No	Yes
Manage staff and PINs	Yes (through the server)	No	No	No
Read the audit trail	Yes	No	No	No

Notes:

- Recipes can be edited directly by owners and cooks. Saving a new recipe version through the version function is owner only. [C]
- A cashier can void or refund only a sale they took themselves. A void is allowed only on the day of the sale. After that, use a refund. [C][R]
- A cook or a purchaser can submit a stock count, but it stays "pending" until an owner approves it. An owner's own count applies immediately. [C][R]
- Staff PINs are checked only on the server. The PIN hash and salt cannot be read or written from the browser by any role. [C][R]

Part 3. Area by area: the rules

3.1 Sales at the till

What can be sold. Only dishes on the current menu, at the current menu price. The price comes from the database, not from the screen. [C][R]

Payment methods.

- **Cash.** The whole total is cash.
- **Transfer.** The whole total is a bank transfer. Automatic confirmation through Monnify exists, but is switched off by default for each shop and has **not** been proved end to end. [C][S]
- **Split.** Part cash and part transfer. The two parts must add up exactly to the total, and each must be more than zero.
- **Credit.** The customer owes the money. The sale creates a customer debt (see 3.2).

Checks on every sale. The person must be a cashier, owner or Supa Admin. The plan must be active. There must be at least one item, each with a quantity above zero. The total must be above zero. Cash and transfer must add up exactly to the total. [C][R]

Stock. For dishes set to "made to order", each plate sold reduces the stock of its ingredients, using the recipe. Dishes set to "cooked in batches" reduce stock when a batch is logged instead. [C][R]

Voids and refunds.

Type	Rule
Void	Same day only. The whole sale is cancelled and its ingredients go back into stock. Not allowed on a part-refunded sale.
Part refund	More than £ 0 and less than what is left on the sale.
Full refund	Refunds what is left.

Every void or refund needs a reason of 5 or more characters, and is saved as its own record with who did it and why. The sale's status changes, but the record of the refund is never edited. [C][R]

3.2 Customer credit (money customers owe)

Each debt has an amount, a running total of what has been paid, and a running total of what has been written off. These totals are worked out from the entries. Nobody types them. [C][L]

Entry	Who	Rule
Payment	Cashier, owner	Cash or transfer. Part-payments are allowed. A payment can never be more than what is still owed.
Write-off	Owner only	Partial or the whole balance. Reason required. Shown as "Written off", never as "Paid".
Reversal	Owner only	Undoes one payment or one write-off, once, for exactly the same amount. Reason required. A reversal cannot be reversed.

Entry	Who	Rule
Debt by hand	Owner only	For money owed that did not come from a till sale. Cashiers create credit only by selling on credit at the till.

A debt is **settled** when nothing is left to pay (amount minus paid minus written off is zero). The size of a debt can never be changed after it is created. [C][L][R]

Debts that were marked settled before this system existed were given one "carried over" payment for their full amount. It is not known whether the money was actually received, and the history says so. [C]

3.3 Catering

A catering order has a contract total, a deposit, and later payments. Each deposit and each payment is its own entry. [C][L]

- **Reversal:** owner only, with a reason, for exactly the amount of the entry, once.
- **Payments** record whether they were **cash** or **transfer**.
- **Deposits taken when the order is created do not record a method.** The system therefore cannot tell whether a deposit was cash, and deposits are **not** counted in the cash drawer (see 4.1). This is a known gap (Part 9).
- Orders carry a status (the statuses include enquiry, confirmed, delivered and cancelled). It is changed through a database function that cashiers, owners and Supa Admins can use. [C]

3.4 Supplier ledger (money you owe suppliers)

What you owe a supplier is worked out from entries only. [C][L]

Balance owed = credit purchases + payments that were reversed – payments – credit purchases that were reversed.

- A **negative** balance means the supplier owes you. This happens if you pay more than you owe (an advance). The screen asks for confirmation first: "This is more than you owe. The supplier will owe you the difference." [C][L][S]
- **Payment:** purchaser, owner. **Reversal of a payment:** owner only, with a reason.
- **Reversal of a credit purchase** (when a purchase is reversed, see 3.5) lowers the balance. If the supplier has already been paid part of it, the balance can go negative. The screen warns: "This reversal leaves the supplier owing you ₦X." [C][L][R]
- Supplier payments do **not** record cash or transfer. This is why they are not part of the cash drawer (Part 9).

3.5 Purchases, prices and stock

Logging a purchase. The purchaser or owner chooses the ingredient, quantity, unit, amount paid, how it was paid (cash, transfer or credit), a **grade** (A, B or C) and a **season** (plenty, normal or scarce). In one step the database: [C][R]

1. Converts the quantity into the ingredient's base unit (for example kg).
2. Works out the new price per base unit (amount paid divided by base quantity).
3. Updates the ingredient's current price, previous price, grade, season and stock.
4. Updates the price saved for that grade.

5. Saves the purchase, including a **snapshot of what the ingredient looked like just before** (price, grade, season, grade price) and how much stock this purchase added.
6. If it was bought on credit from a supplier, adds a supplier debt entry.
7. If the new price is more than 5% above the price for that grade, raises a price-spike alert for the owner.

Reversing a purchase (owner only, reason required). A purchase can be reversed only if **nobody has changed that ingredient's price since** (which also means it is the latest purchase of that ingredient). If a newer purchase or price change exists, the screen says: "Cannot be reversed because a newer price or purchase exists for this ingredient." [C][L][R]

A reversal:

- puts the ingredient back to the price, grade and season it had before, from the snapshot;
- puts the grade price back (or removes it if the purchase created it);
- takes the stock back out. **Stock is allowed to go below zero**, with a warning, because the kitchen may already have used it;
- lowers what you owe the supplier, if it was a credit purchase;
- adds a reversal line (negative quantity and amount) beside the original and an audit entry.

Purchases saved **before 2 October 2026** have no snapshot and can never be reversed. [C][R]

An alert raised by a purchase stays after the purchase is reversed. It records what happened. [C]

Who can change stock and prices. Only the approved paths: purchases, reversals, sales, voids, wastage, batches, stock counts and the price-change function. **Nobody, including owners, can edit an ingredient's stock, cost, grade, season or price date directly.** An ingredient's unit cannot be changed once it has any purchase, stock history or stock-count line, and an ingredient with history cannot be deleted. [C][R]

Stock history. Every change to a stock figure writes a line to the stock trail with the reason. The reasons are: bought, purchase reversed, used in a sale, sale voided and put back, wastage, wastage entry removed, used in a batch, batch reversed and put back, stock-take correction, opening count, starting figure, and "changed by hand". "Changed by hand" should always be zero. As of 2 October 2026 it is zero. [C]

3.6 Wastage, batches and stock counts

- **Wastage:** cooks, purchasers and owners can log it. The database reduces stock. Owners can remove a wastage entry, which puts the stock back and is itself recorded in the stock trail. Wastage entries are a plain log, not a ledger, and an owner can edit or delete them. [C][R]
- **Batches:** cooks and owners log a batch of a dish that is set to "cooked in batches". The batch reduces stock for each ingredient. A dish set to "made to order" cannot have batches. Logging a batch was failing for everyone until 3 October 2026 because of a database fault (FO); the fix is applied and a batch of a normal dish was logged successfully in the rehearsal. A batch cannot be edited or deleted by anyone signed in. **A mistake is corrected by an owner reversing the batch** (reason of 5+ characters): the full ingredient quantities go back into stock, even if some has been used since, and the batch no longer counts in any total. A batch recorded before 3 October 2026 carries no stock trail and cannot be reversed. [C][R]
- **Stock counts:** counted amounts are compared with what the system expected. A difference needs a note. A cook's or purchaser's count waits for an owner to approve. Approval applies the counts as "stock-take correction" lines. [C][R]

3.7 Cash drawer (shifts)

A **shift** is one cashier's period at the till, with an opening float and a closing count.

- **One open shift per business at a time.** A second one cannot be opened until the first is closed. A handover is: close, then open. [C][L][R]
- **Opening** goes through a database function. **Closing** is done by the server, which works out the expected cash (see 4.1) and saves how it was worked out: cash sales, catering cash, debt cash. [C][L]
- If the count differs from the expected cash, an alert for the owner is raised. [C]
- **A closed shift can never be changed or deleted by anyone signed in.** [C][L][R]
- **Correcting a wrong count:** an owner adds an **adjustment** (reason required). The original count stays. The screen shows the original, each adjustment, and the adjusted result. The count can never be adjusted below ₦0. [C][L][R][S]
- **A shift left open:** an owner can close it with a reason. The expected cash is worked out up to that moment. The owner may leave the count empty, in which case the shift shows "not counted", raises no shortage alert, and cannot be adjusted. [C][L][S]

3.8 Channel payouts and price decisions

- **Channel payouts** (money received from a delivery platform) are recorded by the owner. The database works out gross sales for the period and compares with what was received, and raises an alert for any difference. [C]
- **Price decisions** (publish, adjust portion, defer) are recorded by the owner. Publishing changes the dish's price. [C]
- Both tables are frozen: they can only be written by the app's own functions (`log_channel_payout` , `decide_price`) and cannot be edited or deleted by anyone signed in. [C][R]
- **Correcting a payout:** an owner reverses it (reason of 5+ characters). The reversal cancels the payout, which then shows as "Reversed" and no longer counts. The owner records the correct payout on the normal screen. A payout's mismatch alert is not linked to the payout, so the owner dismisses it by hand. [C][R][S]
- **Correcting a price decision:** an owner reverses it (reason of 5+ characters). For a "publish" decision the dish price goes back to what it was, but only if the dish still has the price that decision set and no later published decision is still standing. If the price has changed since, including by a hand edit to the dish, the reversal is refused. Adjust-portion and defer decisions changed no price, so only the record is reversed. [C][R][S]
- Each entry can be reversed once. A reversal is final and cannot itself be reversed. [C][R]

Part 4. How the key numbers are worked out

4.1 Expected cash in a drawer

Expected cash = opening float + cash part of every paid or part-refunded cash or split sale in the shift – part refunds on those sales (never more than the cash taken on that sale) + cash catering payments in the shift + cash debt payments in the shift

- Voided and fully refunded sales add nothing. [C]
- A catering or debt payment that is reversed has a matching minus entry with the same method, so a payment and its reversal inside the same shift net to nothing. A reversal made in a later shift counts in that later shift. [C][L]
- The shift window runs from the moment the shift opened to the moment it closed. [C]
- **Not counted:** catering deposits (no method saved), cash purchases, supplier payments, and the transfer part of any sale. [C]
- The count the cashier types is compared with this number. **Difference = counted – expected.** After an owner adjustment, the adjusted difference = counted + adjustments – expected. [C][L]
- The calculation lives in one place (`src/lib/cash-drawer.ts`) and is used by closing a shift, by the owner's cash forecast, and by closing an abandoned shift. [C]

4.2 What a customer owes

Owed = debt amount – payments – write-offs (payments and write-offs both after reversals). Never below zero.

4.3 What you owe a supplier

See 3.4. A negative result means the supplier owes you.

4.4 Ingredient price and plate cost

- **Price per base unit** from a purchase = amount paid ÷ quantity in the base unit. [C]
- **Plate cost** of a recipe = for each ingredient, its recipe quantity (converted to the base unit) × its price, added up, divided by the number of portions the recipe makes. If the recipe is set to a cost grade, the price for that grade is used; otherwise the ingredient's current price. If a quantity cannot be converted to the ingredient's unit, no cost is given. [C]
- Each sold line keeps the plate cost at the time of sale. [C]

4.5 Profit and loss

- **Gross sales** (paid orders), **cost of goods** (recipe cost of plates sold, plus wastage), **gross margin** and **food cost percentage** come from one calculation (`src/lib/pnl.ts`). [C]
- **Cost frozen at the moment of sale.** Since 1 October 2026 every sold line stores what one plate cost at that moment (`order_items.cost_per_plate_kobo` , set by the database from the recipe version and its cost grade). A signed-in person cannot change it afterwards ("The cost of a sold item cannot be changed"). The profit and loss uses that stored figure as it is, so later price rises do not rewrite earlier sales. [C]

- **Known limitation:** sold lines from before 1 October 2026 carry no stored cost. They are costed with each ingredient's **current** price, and the result counts them as "estimated". In the live data on 3 October 2026, 107 of 110 sold lines are in this group, because almost all are older test sales; all 3 lines since the change carry a frozen cost. Ingredient prices themselves are still not versioned, so a restated cost for old sales is not possible. Wastage entries store their own cost when logged. [C]

Part 5. Audit: what is recorded

5.1 The audit trail

Every important action writes a line to the audit trail with: **who** (the person and their role), **what** (the action), **which record**, a short description, and **when**. The audit trail can be read only by owners and Supa Admins of that business. [C]

Audit lines are written by database functions, by database triggers, or by the server. [C]

Events the system writes (Appendix F lists them all):

Area	Events
Sales	order adjusted (void or refund)
Customer credit	credit created by hand, payment recorded, written off, entry reversed
Catering	order created, payment recorded, payment reversed
Supplier	payment recorded, payment reversed
Purchases and stock	cost changed, purchase reversed, stock count submitted, approved, rejected
Cash drawer	drawer opened, shortage or overage at close, count adjusted, shift closed by owner
Prices	price published
Business and staff	business created, approved, rejected, suspended, reactivated; staff created, role changed, deactivated, PIN reset; sign-in success, failure, account locked
Platform	settings changed, subscription payment recorded, payment connection, payment mode changed

On 2 October 2026 the live audit trail holds 118 lines across 16 event types. [C]

5.2 What is not audited

Being honest about gaps matters more than a long list:

- **A purchase being logged** has no audit line of its own. The purchase record is the evidence, and a price change writes a "cost changed" line.
- **Wastage and batch entries** have no audit line.
- **A shift closing with no difference** has no audit line. The shift record itself holds the figures. Opening a shift does.
- **Sales** are recorded in the orders themselves, not in the audit trail.

5.3 Protection of the audit trail itself

- Nobody can edit or delete an audit line. [C][R]
- Nobody signed in can *add* an audit line either. Lines are written only by database functions and server routes, so a cook can no longer pad or forge the trail (this was gap A2, closed on 3 October 2026). [C][R]

Part 6. Every kind of money event, accounted for

For each kind of transaction: where it is recorded, what writes it, who may do it, how it is undone, what audit line it leaves, and whether it counts in the cash drawer's expected cash.

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
1	Cash sale	orders, order lines, stock trail	create_cash_order	Cashier, owner	Void (same day) or refund	none for the sale; "order adjusted" when voided	Yes
2	Split sale	same	create_cash_order	Cashier, owner	same	same	Cash part only
3	Transfer sale	orders, payment requests	create_transfer_order ; provider webhook marks it paid	Cashier, owner; server	Void or refund	same	No
4	Credit sale	orders + customer debt	create_credit_order	Cashier, owner	Debt write-off or reversal	credit events	No
5	Void	order adjustments, order status, stock put back	adjust_order	Cashier (own), owner	Not reversible: it is final	order adjusted	Removes the sale
6	Part or full refund	order adjustments, order status	adjust_order	Cashier (own), owner	Final	order adjusted	Reduces cash by the refund, up to the cash taken
7	Catering order and deposit	catering order, catering payment (deposit)	create_catering_order	Cashier, owner	Reverse the deposit entry	catering order created	No (no method saved)
8	Catering payment	catering payments	record_catering_payment_v2	Cashier, owner	Owner reversal	catering payment recorded or reversed	Yes, if cash
9	Customer debt payment	credit payments	record_credit_payment	Cashier, owner	Owner reversal	credit payment recorded	Yes, if cash
10	Debt write-off	credit payments	write_off_credit	Owner	Owner reversal	credit written off	No
11	Debt by hand	customer credits	create_manual_credit	Owner	Write-off	credit created by hand	No
12	Supplier credit purchase	supplier entries, purchase	log_purchase	Purchaser, owner	Purchase reversal	none (price change line)	No

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
13	Supplier payment	supplier entries	record_supplier_payment	Purchaser, owner	Owner reversal	supplier payment recorded or reversed	No (no method saved)
14	Cash or transfer purchase	purchases, stock, price	log_purchase	Purchaser, owner	Purchase reversal	cost changed	No
15	Purchase reversal	purchases, stock, price, supplier entry	reverse_purchase	Owner	Final	purchase reversed	No
16	Price change	ingredient price, grade price	set_ingredient_price	Purchaser, owner	Another price change	cost changed	No
17	Wastage	wastage log, stock trail	direct entry; database adjusts stock	Cook, purchaser, owner	Owner removes the entry	none	No
18	Batch	batches, stock trail	log_batch	Cook, owner	Owner reversal (event 29)	none	No
19	Stock count	stock counts, lines, stock trail	submit_stock_count , decide_stock_count	Cook, purchaser, owner; owner approves	A new count	stock count submitted, approved, rejected	No
20	Shift open	cash drawers	open_cash_drawer	Cashier, owner	n/a	drawer opened	n/a
21	Shift close	cash drawers	server route	Cashier (own), owner	Count adjustment	shortage or overage	n/a
22	Shift closed by owner	cash drawers	server route	Owner	Count adjustment (if counted)	shift closed by owner	n/a
23	Count adjustment	cash drawer adjustments	adjust_closed_drawer	Owner	Another adjustment	count adjusted	n/a
24	Channel payout	channel payouts, alert	log_channel_payout	Owner	Owner reversal (event 27)	none	No
25	Price decision	price decisions	decide_price	Owner	Owner reversal (event 28)	price published	No
26	Subscription payment	subscription payments	server only	Platform	n/a	subscription payment recorded	n/a

#	Event	Recorded in	Written by	Who	How it is undone	Audit event	In expected cash?
27	Payout reversal	channel payouts (negated row)	reverse_payout	Owner	Final	payout reversed	No
28	Price decision reversal	price decisions; dish price	reverse_price_decision	Owner	Final	price decision reversed	No
29	Batch reversal	batches (negated row), stock trail	reverse_batch	Owner	Final	batch reversed	No

Events that do not fit the table: staff changes and PIN resets (server routes, audited); business sign-up and approval (audited); daily summary emails and reminders (system, logged separately). [C]

Part 7. The security model in plain words

Two ways in. The app talks to the database in two ways. [C]

1. **From the browser**, using the person's login. The database checks the login on every request using row security: a person sees and changes only their own business's rows, and only what their role allows.
2. **Through server routes**, which check the login token or a secret first and then use a stronger key. These are used where the browser must not be trusted: signing in with a PIN, closing a shift, managing staff, connecting a payment provider, the scheduled emails and the payment webhook. Appendix E lists all 16.

The locks, in layers.

- **Row security** on all 47 tables (45 app tables and 2 private backup tables). No rule lets a signed-out visitor read or write anything. [C][R]
- **Write functions.** Money tables, refund records, the audit trail, payouts, price decisions and batches have no direct write access for signed-in people. Only database functions, which check the role, the business and the plan, can write. [C][R]
- **Guard triggers.** Even where a write path exists, a trigger on the table refuses edits and deletes of finished records, and refuses direct writes to protected columns. [C][R]
- **Fixed search path.** All 65 database functions that run with extra privilege have their search path fixed, so they cannot be tricked into using a different table. [C]
- **Secrets** (payment provider keys, the scheduler secret) are stored in the database vault and never returned to the browser. [C]

Staff sign-in. PINs are hashed. The server is the only place a PIN is checked. A signed-in person's role and business come from the server and cannot be set from the browser. Businesses must be approved. A business that is suspended or whose plan has ended is locked out. Owners can still sign in to see the locked screen. [C]

What the controls do not cover is listed in Part 9.

Part 8. Evidence: what has actually been proved

8.1 Automated tests

- 236 automated checks across 25 test files pass on the current code. They cover the shared calculations (balances, reversals, expected cash, adjustments, labels and rules shown on screens). [L]
- For every step of the ledger programme, the database changes were tested on a local copy of the database before release: the loophole shown first, then every allowed action, every refusal, other businesses, every role, repeated runs, and the rollback. [L]

8.2 The live rehearsal, 2 and 3 October 2026

A script acted as each role in the Demo Kitchen business, using the real database functions and rules, and then always ended with a deliberate error so that nothing was saved. [R]

- **First run, 2 October (127 steps):** 107 passed, 9 were flagged, 11 were information only. The flags were six genuine gaps (A1, A2, A3, A4), two steps caused by the batch fault (F0), and one false alarm (the test set a business status to the value it already had).
- **Run after F0, B1 to B3, A1 and A2 were applied:** 114 passed, 6 flagged, 0 test errors. The 6 were exactly A3 and A4.
- **Final run, 3 October, after A3 and A4 were applied:** 119 passed, 1 flagged, 0 test errors, plus information rows. The one flag was again the false alarm of setting a status to its current value. The test was corrected to use a different value, and that step was re-run: an owner trying to change their own business status is refused ("Only the platform admin can change a business's status").
- **Passed (examples):** a cashier took a cash sale and voided it; part and full refunds worked and over-refunds were refused; cooks, purchasers and visitors were refused where they should be; a purchaser logged a credit purchase and an owner reversed it, restoring the price and the supplier balance; supplier, customer-credit and catering payments and reversals followed the role rules; stock counts needed owner approval; shifts opened once and could not be opened twice; closed shifts could not be rewritten; an owner could add a count adjustment; another business could see none of Demo Kitchen's data and could not act on it; visitors could read and write nothing; no owner could write, edit or delete refund records, audit lines, payouts, price decisions or batches directly; the real functions for a price decision and for a batch of a normal dish still worked.
- **By design, still refused:** a batch of a "made to order" dish; automatic transfer sales in a business that has not switched them on.
- **Not saved:** the rehearsal changed nothing.
- What the rehearsal cannot show: it tests the database as each role, not a person using the screens.

8.2b The correction functions, 3 October 2026

- **Local database copy:** refusals for a cashier, cook, visitor, another business's owner and an expired plan; short reasons; double reversals; forged reversal rows; the price chain (reversing an older decision is refused while a newer one stands, and works after the newer one is reversed); a price edited by hand blocks reversal; stock put back in full after later use; a batch whose dish was later made "made to order" can still be reversed; rollback and re-apply. [L]
- **Live rehearsal of the correction functions:** 37 passed, 0 findings, 0 test errors. Stock went 5.20 to 5.19 on logging a batch and back to 5.20 on reversal. The data was checked afterwards and nothing was left behind. [R]

-
- **Full rehearsal with the correction steps included:** 143 passed, 0 findings, 0 test errors. [R]
 - 246 automated checks pass (10 are new for the corrections). [L]

8.3 What is not yet proved

- **The new screens have not been used by a person.** Customer credit, purchase reversal, the ingredient lock, the cash drawer screens and the three correction screens (Payouts, Pricing review history, Recent batches) passed their database tests and were released, but a person using them in Demo Kitchen has not been reported. The Cloudflare build for the release (commit `2862f2f`) was not confirmed by this document's author. [S]
- **Monnify automatic transfer confirmation** has not been tested end to end with real keys. [S]
- **The real stock-take, wastage and sale functions** were exercised by the live rehearsal for the sale, void, refund, wastage and stock-count paths. The batch path now works for a dish that is not set to "made to order". [R]

Part 9. Known gaps and open items

Ranked by what matters most. Each says what it is, what could go wrong, and the status.

Faults and gaps in the controls

Ref	What	Risk	Status
F0	Logging a batch fails for everyone. The trigger that stamps the recipe version on batches also tries to set a plate cost, which batches do not have. Error: <code>record "new" has no field "cost_per_plate_kobo"</code> . Confirmed live on 2 October 2026. No batch has been logged since 25 September.	Cooks cannot log batches. Stock is not reduced for batch dishes.	Closed 3 October 2026. Fix applied (<code>20261025_batch_trigger_fix.sql</code>) and a batch was logged in the rehearsal.
A1	Refund records (<code>order_adjustments</code>) can be written directly by cashiers and owners, and edited by owners.	A forged part refund lowers expected cash and can hide a drawer shortage.	Closed 3 October 2026. Direct writes refused, edits blocked.
A2	Audit lines can be added directly by any signed-in staff.	The audit trail can be padded or forged.	Closed 3 October 2026. Direct writes refused.
A3	Channel payouts can be written and edited directly by owners.	Payout figures can be altered after the check.	Closed 3 October 2026. Frozen; only <code>log_channel_payout</code> writes.
A4	Price decisions and batches can be written, edited or deleted directly by owners.	Decision history and batch history can be altered.	Closed 3 October 2026. Frozen; only <code>decide_price</code> and <code>log_batch</code> write. Corrections need a future function.
B1 to B3	Visitors still hold table privileges (blocked by row security), signed-in users hold TRUNCATE, TRIGGER and REFERENCES, and visitors can be given execute on trigger functions. None is reachable through the app.	Defence in depth only.	Closed 3 October 2026. Visitors hold no table privileges, signed-in users no longer hold TRUNCATE, TRIGGER or REFERENCES, and visitors cannot run any trigger function. Backups of the old grants are kept in two private tables.
B5	The sign-in screen needs to show who can sign in, so one server action (<code>list_staff</code>) returns the id, display name, role and active flag of a business's active staff to anyone who knows the business code , without signing in. Business codes are short names such as <code>demo-kitchen</code> . It never returns PINs, phones or emails.	An outsider can list staff names and roles of a business whose code they know, and then try PINs. Repeated wrong PINs lock the account (failed attempts and a lock time are saved, and one account lock is in the live audit trail). I have not tested the lockout limits.	Design choice. Needs a decision.

Ref	What	Risk	Status
B4	<code>business_has_access</code> , <code>catering_enabled</code> , <code>payment_mode_of</code> and <code>trial_limits_apply</code> accept a business id from the caller. A signed-in person who guesses another business's id can learn whether it is active, has catering on, its payment mode, or is on a trial.	Low. No money or customer data.	Accepted and documented. These functions are needed by the access rules.

Cash-drawer accounting gaps

- **Catering deposits** are not counted in expected cash because no cash or transfer method is saved for them. Fix: add a method to the deposit. Not built.
- **Cash purchases and supplier payments** are not taken out of expected cash. Supplier payments record no method. A "cash paid out of the drawer" entry is planned for later.
- **A shift closed by an owner without a count** has no difference and cannot be adjusted.

Other open items

- **Correction limits.** The six batches logged before 3 October 2026 cannot be reversed (no stock trail). A payout's mismatch alert is not linked to the payout and must be dismissed by hand. A dish price edited by hand is not timestamped, so the price guard on reversing a decision compares prices only.
- **Stale shift.** The 25 September shift in Demo Kitchen was closed by the owner on 3 October 2026 (reported by the owner; the database shows no open shifts). [C]
- **PIN hashing.** The upgrade (a stronger scheme with a secret pepper and re-hashing at sign-in) is planned and not built.
- **Public prices for visitors.** Prices are shown to signed-in people only. Showing them to signed-out visitors needs a decision.
- **Expiry reminder emails** exist but are switched off and not scheduled.
- **Backups.** Database backups and point-in-time recovery should be confirmed in the Supabase dashboard. This document cannot confirm them.
- **No staging environment.** Changes are tested locally and then run on the live database.
- **Older sales are costed at today's prices.** Sold lines from before 1 October 2026 have no frozen cost (Part 4.5). Sales since then are frozen.
- **Other deletes left open by decision:** suppliers, recipes, wastage entries, unit conversions and alerts can still be edited or deleted by owners. Each is a configuration or log table, not a ledger.
- **Recipe price edits.** An owner can change a dish's price directly without a price decision record. Sales keep the price they were sold at.

Part 10. How the ledger programme got here

Step	What	Released	SQL files
Till price path	Cash and split sales saved by the database, which reads menu prices	PR #36	20261015
Locks	Catering table lock, sales delete lock, ledger delete lock	2 Oct	20261016 to 20261018
1	Catering payment ledger	PR #37	20261019
2	Supplier ledger	PR #38	20261020
3	Customer credit ledger	PR #39	20261021
4	Purchase ledger and reversals	PR #40	20261022
4b	Ingredient stock, price, unit and delete lock	PR #41	20261023
5	Closed cash drawers, adjustments, force close	PR #42	20261024
6	Security and safety sweep: batch fix, permission clean-up, locks on refund records, audit trail, payouts, price decisions and batches	PR #43 (SQL applied live 3 Oct)	20261025, 20261026
7	Owner corrections for payouts, price decisions and batches	PR #44 (SQL applied live 3 Oct)	20261027

Each step shipped as: database change, a check query, a rollback, a tested local run, and a verified release.

Part 11. Sign-off checklist for the security and safety sweep

#	Check	Result on 3 October 2026
1	Every table has row security	Done: 47 of 47 [C]
2	No rule opens any table to signed-out visitors	Done: 0 rules [C]
3	Every privileged function has a fixed search path	Done: 65 of 65 [C]
4	Every function that moves money checks role and business	Done: every writer function listed in Part 6 checks the role in its body and filters by the caller's business [C][R]
5	Money tables cannot be written directly	Done for sales, catering, credit, supplier, purchases, drawers, stock, refund records, the audit trail, payouts, price decisions and batches [C][R]
6	Finished records cannot be edited or deleted	Done for all ledgers, refund records, the audit trail, payouts, price decisions and batches [C][R]
7	Each business sees only its own data	Done: 6 table reads and 4 actions tested against another business [R]
8	Staff secrets cannot be read or written from the browser	Done [C][R]
9	Every kind of money event is accounted for	Done: Part 6 lists 29
10	Every core path works end to end, including the three corrections	Done at database level: sale, void, refund, purchase, reversal, payments, stock count, shift, batch and price decision all ran in the rehearsal [R]
11	New screens used by a person in Demo Kitchen (including the three correction screens)	Not done [S]
12	Backups confirmed	Not done: needs the Supabase dashboard

Sign-off is not yet possible. The database controls are in place, but two items are outstanding and both need a person: the Demo Kitchen walk-through on the screens (check 11) and confirmation of backups (check 12). The stale 25 September shift should also be closed.

Technical appendices

All appendices were read from the live database and the code on 2 and 3 October 2026 unless stated. They are for engineers and auditors.

Totals on 3 October 2026: 47 tables (45 app tables and 2 private grant-backup tables), 67 access rules (policies, 29 of them write rules, none on the money tables), 85 functions (53 callable, 32 trigger functions, 65 of them with extra privilege, all with a fixed search path), 46 triggers, 69 foreign keys, 5 scheduled jobs, 2 vault secrets.

Appendix A. Tables and who can write to them

"Direct" means a signed-in person writing to the table from the browser. **A dash means the browser cannot** (writes go through functions or the server). "Any member" means any signed-in person of the business. Read access is always limited to the person's own business.

Table	Kind	Read	Direct insert	Direct update	Direct delete
archived_app_state_snapshots	Archived	nobody	-	-	-
archived_saved_recipe_configs	Archived	nobody	-	-	-
audit_logs	Audit trail	owner, supa_admin	-	-	-
batches	Production log	cook, owner, supa_admin	-	-	-
business_features	Settings	any member	-	-	-
business_news_prefs	Settings	any member	-	-	-
business_payment_settings	Settings	owner, cashier, supa_admin	-	-	-
businesses	Tenant record	own business (members); all (platform admin)	owner, supa_admin (own row)	owner, supa_admin (own row; billing and status columns guarded by a trigger); platform admin	owner, supa_admin (refused while ledger records exist)
cash_drawer_adjustments	Ledger	owner, cashier, supa_admin	-	-	-
cash_drawers	Ledger	owner, cashier, supa_admin	-	-	-
catering_deposits	Catering order	owner, cashier, supa_admin	-	-	-
catering_order_items	Catering order	cook, owner, cashier, purchaser, supa_admin	-	-	-
catering_payments	Ledger	owner, cashier, supa_admin	-	-	-

Table	Kind	Read	Direct insert	Direct update	Direct delete
catering_reminder_log	System log	nobody (server only)	-	-	-
channel_payouts	Payout record	owner, supa_admin	-	-	-
contact_messages	Platform	platform admin	- (server)	platform admin	-
credit_payments	Ledger	owner, cashier, supa_admin	-	-	-
customer_credits	Ledger	owner, cashier, supa_admin	-	-	-
daily_summary_log	System log	nobody (server only)	-	-	-
ingredient_grade_prices	Price record	cook, owner, cashier, purchaser, supa_admin	-	-	-
ingredients	Stock and price	cook, owner, cashier, purchaser, supa_admin	owner, purchaser, supa_admin (stock and price columns must start at zero)	owner, purchaser, supa_admin (stock, cost, grade, season, price date locked; unit locked once history exists)	owner, supa_admin (refused once history exists)
margin_flags	Alerts	by the role the alert is for	owner, supa_admin	by role (to mark seen)	owner, supa_admin
news_feed_status	System	nobody	-	-	-
news_items	Reference	cook, owner, cashier, purchaser, supa_admin	-	-	-
order_adjustments	Sales record (voids, refunds)	owner, cashier, supa_admin	-	-	-
order_items	Sales	owner, cashier, supa_admin	-	-	-
orders	Sales	owner, cashier, supa_admin	-	-	-
payment_events	System	nobody	-	-	-
payment_requests	Payments	owner, cashier, supa_admin	-	-	-
platform_settings	Platform	nobody directly (read through public_settings)	-	-	-

Table	Kind	Read	Direct insert	Direct update	Direct delete
price_decisions	Decision log	owner, supa_admin	-	-	-
purchases	Ledger	owner, purchaser, supa_admin	-	-	-
recipe_items	Configuration	cook, owner, cashier, purchaser, supa_admin	cook, owner, supa_admin	cook, owner, supa_admin	owner, supa_admin
recipe_variants	Configuration	owner, supa_admin	-	-	-
recipes	Configuration	cook, owner, cashier, purchaser, supa_admin	cook, owner, supa_admin	cook, owner, supa_admin	owner, supa_admin
staff_users	Staff	owner, supa_admin (8 non-secret columns only)	no privilege	no privilege	-
stock_count_lines	Stock	cook, owner, purchaser, supa_admin	-	-	-
stock_counts	Stock	cook, owner, purchaser, supa_admin	-	-	-
stock_movements	Ledger	owner, purchaser, supa_admin	-	-	-
subscription_payments	Platform	nobody (server only)	-	-	-
subscription_reminder_log	System log	nobody (server only)	-	-	-
supplier_transactions	Ledger	owner, purchaser, supa_admin	-	-	-
suppliers	Configuration	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, supa_admin
unit_conversions	Configuration	cook, owner, cashier, purchaser, supa_admin	owner, purchaser, supa_admin	owner, purchaser, supa_admin	owner, supa_admin
wastage_logs	Log	cook, owner, purchaser, supa_admin	cook, owner, purchaser, supa_admin	owner, supa_admin	owner, supa_admin (stock is put back)

Notes:

- `staff_users` has access rules that look as if owners can insert and update, but signed-in users hold **no** insert or update privilege on any column, so those rules have no effect. PIN hash and salt are not readable. [R]
- Ledger tables also have a guard trigger that refuses any change or deletion by a signed-in person (Appendix C), so a mistaken access rule added later would still be refused.

Appendix B. Functions callable from the app

Every function below fixes its search path. "Definer" functions run with the privileges of their owner and therefore must check the caller themselves; each lists the roles it accepts.

Function	Runs as	Roles accepted	Purpose
<code>open_cash_drawer(float)</code>	definer	owner, cashier, supa_admin	Open a shift (one per business)
<code>adjust_closed_drawer(drawer, amount, reason)</code>	definer	owner, supa_admin	Add a count adjustment to a closed shift
<code>create_cash_order(channel, tier, method, cash, transfer, items)</code>	definer	owner, cashier, supa_admin	Cash, transfer or split sale
<code>create_transfer_order(channel, tier, items)</code>	definer	owner, cashier, supa_admin	Transfer sale
<code>create_credit_order(channel, tier, customer, phone, items)</code>	definer	owner, cashier, supa_admin	Credit sale and debt
<code>cancel_unpaid_order(order, reason)</code>	definer	owner, cashier, supa_admin	Cancel an order that was never paid
<code>adjust_order(order, type, reason, amount)</code>	definer	owner, cashier, supa_admin	Void or refund (cashier: own sales only)
<code>record_credit_payment(credit, amount, method)</code>	definer	owner, cashier, supa_admin	Debt payment
<code>write_off_credit(credit, amount, reason)</code>	definer	owner, supa_admin	Write off
<code>reverse_credit_entry(entry, reason)</code>	definer	owner, supa_admin	Reverse a debt entry
<code>create_manual_credit(customer, phone, amount, note)</code>	definer	owner, supa_admin	Debt by hand
<code>create_catering_order(...)</code>	definer	owner, cashier, supa_admin	Catering order with deposit
<code>record_catering_payment_v2(order, amount, method)</code>	definer	owner, cashier, supa_admin	Catering payment
<code>record_catering_payment(order, amount)</code>	definer	(hands off to v2)	Older name, same checks
<code>reverse_catering_payment(payment, reason)</code>	definer	owner, supa_admin	Reverse a catering entry
<code>set_catering_status(order, status)</code>	definer	owner, cashier, supa_admin	Change catering status

Function	Runs as	Roles accepted	Purpose
<code>log_purchase(...)</code>	defin er	owner, purchaser, supa_admin	Log a purchase with snapshot
<code>reverse_purchase(purchase, reason)</code>	defin er	owner, supa_admin	Reverse the latest purchase
<code>reverse_payout(payout, reason)</code>	defin er	owner, supa_admin	Reverse a channel payout
<code>reverse_price_decision(decision, reason)</code>	defin er	owner, supa_admin	Reverse a price decision; restores the dish price for a published decision
<code>reverse_batch(batch, reason)</code>	defin er	owner, supa_admin	Reverse a batch; puts the stock back
<code>set_ingredient_price(ingredient, price, grade, season)</code>	defin er	owner, purchaser, supa_admin	Change a price
<code>record_supplier_payment(supplier, amount, note)</code>	defin er	owner, purchaser, supa_admin	Supplier payment
<code>reverse_supplier_payment(payment, reason)</code>	defin er	owner, supa_admin	Reverse a supplier payment
<code>log_batch(...)</code>	defin er	cook, owner, supa_admin	Log a batch
<code>submit_stock_count(note, opening, lines)</code>	defin er	cook, owner, purchaser, supa_admin	Submit a count
<code>decide_stock_count(count, approve)</code>	defin er	owner, supa_admin	Approve or reject a count
<code>log_channel_payout(...)</code>	defin er	owner, supa_admin	Record a channel payout and compare
<code>decide_price(recipe, decision, price, by)</code>	defin er	owner, supa_admin	Record a price decision
<code>save_recipe_version(...)</code>	invoker	owner, supa_admin	Save a new recipe version (row security applies)
<code>save_recipe_variant(...), delete_recipe_variant(variant)</code>	defin er	owner, supa_admin	Recipe variants
<code>set_news_enabled(enabled)</code>	defin er	owner, supa_admin	News watch switch
<code>set_payment_mode(mode)</code>	defin er	owner, supa_admin	Manual or automatic transfer confirmation
<code>ingredient_ids_with_history()</code>	defin er	owner, purchaser, supa_admin	Ingredients whose unit is locked

Function	Runs as	Roles accepted	Purpose
<code>ingredient_has_history(id)</code>	defin er	helper	Used by the ingredient guard; own business only
<code>list_businesses_for_review(status)</code>	defin er	platform admin (checked by role)	Business approvals
<code>public_settings()</code>	defin er	any signed-in person	Prices, locked-screen text, expiry banner
<code>business_has_access(id) , catering_enabled(id) , payment_mode_of(id) , trial_limits_apply(id)</code>	defin er	helpers (B4)	Used inside access rules and screens
<code>audit_naira(kobo)</code>	invok er	helper	Formats a naira amount

Server-only functions (no signed-in person can call them): `apply_stock_count` , `attach_payment_account` , `raise_catering_alerts` , `read_payment_connection` , `recipe_plate_cost_kobo` , `record_provider_payment` , `save_payment_connection` , `signup_business` , `supplier_balance_kobo` , `to_base_qty` .

Appendix C. Triggers

Table	Trigger	Function	When	What it does
batches	batches_stamp_version	stamp_batch_version	before insert	Stamps the recipe version only (own function since 3 October 2026, F0 fix)
batches	batches_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts from signed-in people (A4)
batches	batches_no_change	ledger_block_change	before update, delete	Refuses edits and deletes (A4)
batches	batches_stock_mode_check	check_batch_stock_mode	before insert	Refuses batches for "made to order" dishes; labels stock changes "batch use". Skips both for a reversal row
businesses	audit_business_review	audit_business_review	after update	Audit line for status changes
businesses	businesses_status_guard	businesses_status_guard	before insert, update	Only the platform changes status; billing columns only through the server
cash_drawer_adjustments	cash_drawer_adjustments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
cash_drawers	cash_drawers_protect	cash_drawers_protect	before insert, update, delete	Refuses direct open, close, edit, delete by signed-in people
catering_payments	catering_payments_check_reversal	catering_payments_check_reversal	before insert	Reversal rules
catering_payments	catering_payments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
catering_payments	catering_payments_recompute	catering_payments_recompute	after insert	Recomputes the order's cached totals
credit_payments	credit_payments_check	credit_payments_check	before insert	Balance, owner-only and reversal rules
credit_payments	credit_payments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
credit_payments	credit_payments_recompute	credit_payments_recompute	after insert	Recomputes paid, written off, settled
customer_credits	customer_credits_protect	customer_credits_protect	before update	Refuses any direct change by a signed-in person
ingredients	ingredients_log_stock	log_stock_movement	after update	Writes a stock movement with its reason
ingredients	ingredients_protect	ingredients_protect	before insert, update	Locks stock, price, grade, season, price date; locks unit once history exists

Table	Trigger	Function	When	What it does
ingredients	ingredients_protect_delete	ingredients_protect	before delete	Refuses delete once history exists
ingredients	ingredients_trial_limit	enforce_trial_ingredient_limit	before insert	Trial plan limit
ingredients	trg_audit_ingredient_cost	audit_ingredient_cost	after update	"Cost changed" audit line
order_adjustments	order_adjustments_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts (A1)
order_adjustments	order_adjustments_no_change	ledger_block_change	before update, delete	Refuses edits and deletes (A1)
order_adjustments	trg_audit_order_adjustment	audit_order_adjustment	after insert	"Order adjusted" audit line
order_items	order_items_apply_stock	apply_sale_stock	after insert	Reduces stock for "made to order" dishes
order_items	order_items_lock_cost	order_items_lock_cost	before update	Keeps the plate cost fixed
order_items	order_items_stamp_version	stamp_recipe_version	before insert	Stamps recipe version and plate cost
orders	orders_payment_guard	orders_payment_guard	before insert, update	Rules for who can mark an order paid
orders	orders_reverse_stock	reverse_sale_stock	after update	Puts stock back when a sale is voided
price_decisions	price_decisions_no_direct_insert	block_direct_insert	before insert	Refuses direct inserts (A4)
price_decisions	price_decisions_protect	price_decisions_protect	before update, delete	Refuses edits and deletes, except the database emptying the link to a deleted recipe or user (A4)
price_decisions	trg_audit_price_published	audit_price_published	after insert	"Price published" audit line
purchases	purchases_check	purchases_check	before insert	Reversal rules; refuses direct inserts
purchases	purchases_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
recipe_items	recipe_items_trial_limit	enforce_trial_recipe_items_limit	before insert	Trial plan limit
recipes	recipes_cleanup_variants	cleanup_recipe_variants	after delete	Removes variants of a deleted recipe
recipes	recipes_set_dish_id	set_recipe_dish_id	before insert	Groups versions of one dish
recipes	recipes_trial_limit	enforce_trial_recipe_limit	before insert	Trial plan limit

Table	Trigger	Function	When	What it does
stock_movements	stock_movements_alerts	raise_stock_alerts	after insert	Low-stock and below-zero alerts
supplier_transactions	supplier_transactions_check_reversal	supplier_transactions_check_reversal	before insert	Reversal rules
supplier_transactions	supplier_transactions_no_change	ledger_block_change	before update, delete	Refuses edits and deletes
wastage_logs	wastage_logs_apply_stock	apply_wastage_stock	after insert	Reduces stock
wastage_logs	wastage_logs_restore_stock	apply_wastage_stock	after delete	Puts stock back

The shared guard `ledger_block_change` refuses any change or deletion when a signed-in person is acting. The server key and the SQL editor pass, for administration only.

Appendix D. Ledger rules at a glance

Ledger	Entry kinds	Amount	Reversal	Reason	Totals kept on the parent
catering_payments	deposit, payment, reversal	deposit and payment above zero; reversal below zero	One per entry, same order, exact amount, owner only	5+ chars on reversal	deposit and additional payments, recomputed
credit_payments	payment, write_off, reversal	payment and write-off above zero; reversal below zero	One per entry, same debt, exact amount, owner only	5+ chars on write-off and reversal	paid, written off, settled, recomputed
supplier_transactions	purchase_on_credit, payment, reversal, purchase_reversal	above zero	Payment reversal and credit-purchase reversal, once each, owner only	5+ chars on both reversals	none: balance worked out from entries
purchases	purchase, reversal	purchase above zero; reversal below zero (quantity and amount)	One per purchase, same ingredient, exact quantity and amount, owner only	5+ chars	stock and price on the ingredient
cash_drawers	one row per shift	float, count, expected, difference	not applicable	5+ chars when closed by an owner	breakdown saved at close
cash_drawer_adjustments	adjustment	not zero	An adjustment is corrected by another adjustment	5+ chars	adjusted count worked out
channel_payouts	entry, reversal	reversal carries negated figures	One per entry, owner only (reverse_payout)	5+ chars	none: totals sum the rows
price_decisions	entry, reversal	reversal swaps previous and suggested price	One per entry, owner only; price restored only if unchanged and no later published decision stands	5+ chars	dish price
batches	entry, reversal	reversal carries negated yield and costs	One per entry, owner only; full stock put back; needs the batch's stock trail	5+ chars	stock on the ingredients
stock_movements	one per stock change	signed change and balance after	not applicable	reason code	balance on the ingredient

Appendix E. Server routes

All 16 routes run on the server with the service key. Each one checks the caller before doing anything.

Route	Purpose	Who calls it	How the caller is checked
business-signup	New business sign-up	Visitor	Public by design; PIN hashed on the server; business starts "pending"
staff-pin-login	PIN sign-in and the staff list for the sign-in screen	Visitor	Checks the PIN hash; failed attempts lock the account. The staff list needs no sign-in (B5)
staff-admin	Staff, roles, PINs	Owner, Supa Admin	Login token; business and role from the token only
cash-drawer-close	Close your own shift	Cashier, owner	Login token
cash-drawer-force-close	Owner closes a shift left open	Owner, Supa Admin	Login token
payment-connect	Connect Monnify; keys go to the vault	Owner	Login token
payment-start	Ask Monnify for a transfer account	Till	Login token
payment-test	Check saved Monnify keys	Owner	Login token
payment-webhook	Monnify tells NairaPlate money arrived	Monnify	Signature made with the shop's own secret key
platform-admin	Support console	Platform admin	Login token; role from the token only
contact	Website enquiry forwarded to the platform inbox	Visitor	Public by design; payload size limited
daily-summary	Evening summary email	Scheduler	Bearer secret
catering-reminders	Catering reminder emails	Scheduler	Bearer secret
expiry-reminders	Plan expiry emails (off until an admin switches them on)	Scheduler	Bearer secret
news-watch	Reads news feeds for ingredient price headlines	Scheduler	Bearer secret
summary-unsubscribe	"Stop these emails" link	Owner (from email)	Link signed with the summary secret; two steps (view, then confirm)

Scheduled jobs (database scheduler): daily summary 19:00 UTC daily; news watch at minute 20 every hour; catering alerts, catering morning email 05:30 UTC daily; catering evening email 17:30 UTC daily. **Vault secrets:** the site address and the scheduler secret. Expiry reminders are not scheduled.

Appendix F. Audit events

Seen in the live audit trail (count to 2 October 2026): login_success 76, login_failed 15, business_created 5, business_approved 3, cost_changed 3, pin_reset 2, role_changed 2, business_rejected 2, price_published 2, stock_count_submitted 2, security_alert_undelivered 1, account_locked 1, email_undelivered 1, platform_setting_changed 1, contact_message_handled 1, staff_created 1.

Written by database functions and triggers: business_approved, business_created, business_reactivated, business_rejected, business_suspended, catering_order_created, catering_payment_recorded, catering_payment_reversed, cost_changed, credit_created_manual, credit_entry_reversed, credit_payment_recorded, credit_written_off, drawer_count_adjusted, drawer_opened, order_adjusted, payment_mode_changed, price_published, purchase_reversed, payout_reversed, price_decision_reversed, batch_reversed, stock_count_approved, stock_count_rejected, stock_count_submitted, supplier_payment_recorded, supplier_payment_reversed.

Written by server routes: login_success, login_failed, account_locked, staff_created, role_changed, staff_deactivated, pin_reset, drawer_discrepancy, drawer_force_closed, platform_unlock_staff, emergency_owner_pin_reset, emergency_reset_blocked, security_alert_undelivered, email_undelivered, business_approved, business_rejected, business_suspended, business_reactivated, contact_message_handled, subscription_payment_recorded, platform_setting_changed, payment_provider_connected, feature_switched.

Also written by the database function `cancel_unpaid_order` : unpaid_order_cancelled (missing from version 1.0 of this list). Added in version 1.2: payout_reversed, price_decision_reversed and batch_reversed (proved by the rehearsal).

Event types that no one has triggered yet in the live data (for example credit_payment_recorded, drawer_opened, purchase_reversed) are proved by the rehearsals on 2 and 3 October 2026, where each one was written and then undone. [R]

Appendix G. SQL files for the programme

All files are in `supabase/external/`. Each step has the change, a check query and a rollback. Run order matters: a "part A" before its screen is released, a "part B" after.

Files	What
20261015_till_price_a / b	Till price from the menu; direct sale writes closed
20261015_drop_unused_tables, archive_unused_tables	Unused tables removed or archived
20261016 to 20261018	Catering table lock, sales delete lock, ledger delete lock
20261019_catering_payments	Step 1
20261020_supplier_ledger_a / b	Step 2
20261021_credit_ledger_a / b	Step 3
20261022_purchase_ledger_a / b	Step 4
20261023_ingredient_guard	Step 4b
20261024_cash_drawer_ledger_a / b	Step 5
20261025_batch_trigger_fix	F0 fix. Applied 3 October 2026
20261025_hygiene_batch	B1 to B3. Applied 3 October 2026
20261026_a1_refund_records_lock	A1. Applied 3 October 2026
20261026_a2_audit_trail_lock	A2. Applied 3 October 2026
20261026_a3_channel_payouts_lock	A3. Applied 3 October 2026
20261026_a4_decisions_batches_lock	A4. Applied 3 October 2026
20261027_owner_corrections_a	Step 7: owner corrections (one script, no part B). Applied 3 October 2026
system_health_check.sql	Read-only health check of the locks and permissions (version 1.2)
rehearsal_demo_kitchen.sql	The self-undoing rehearsal used in Part 8

Appendix H. Glossary

Word	Meaning
Kobo	One hundredth of a naira. All money is stored in whole kobo.
Ledger	A record where lines are only added, never changed.
Reversal	A new line that cancels an earlier line without erasing it.
Adjustment	A new line that corrects a closed shift's count without changing it.
Row security	The database rule that decides which rows a person can see or change.
Security definer	A database function that runs with extra privilege, so it must check who is calling.
Trigger	A database rule that runs automatically when a row is added, changed or deleted.
Snapshot	What an ingredient looked like just before a purchase, saved with the purchase so it can be reversed.
Base unit	The unit an ingredient is stored in (for example kg).
Expected cash	What the drawer should hold, worked out from the records.
Rehearsal	A test run on the live database that always undoes itself.